Redis Internal Event Loop Architecture

This section explains **how Redis internally accepts TCP connections, registers sockets in epoll, waits for events, reads commands, and processes them**.

This is the heart of why Redis is extremely fast.

Big Picture Architecture

Redis internally follows:

```
None
Single Main Thread

        +

Event Loop

        +

epoll()

        +

Non-blocking TCP sockets
```

Instead of:

```
None
1 thread per client
```

Redis uses:

```
None
1 event loop handling thousands of clients
```

Full Startup Lifecycle

When Redis starts:

```
None
main()

 → Register socket backend

 → Create event loop

 → Create listeners

 → Register accept handlers

 → Enter infinite event loop
```

==========

PART 1 — Startup Begins in main()

==========

```C/C++
int main() {

   ...

   connTypeInitialize();

   initServer();

   initListeners();

   aeMain(server.el);

}
```

What Each Line Means

`connTypeInitialize();`

Loads networking backends.

Registers:

```None
CT_Socket
```

This tells Redis:

```
None
How to read/write TCP sockets
```

initServer();

Creates server structures.

Important:

```
None
server.el = event loop
```

initListeners();

Creates listening sockets:

```
None
0.0.0.0:6379

127.0.0.1:6379
```

```
aeMain(server.el);
```

Starts infinite event loop.

Redis now becomes live server.

==========

PART 2 — Connection Type Registration

==========

What is CT_Socket?

C/C++

```
static ConnectionType CT_Socket = {
    .read = connSocketRead,
    .write = connSocketWrite,
    .accept = connSocketAccept,
    .listen = connSocketListen,
    .set_read_handler = connSocketSetReadHandler,
    .set_write_handler = connSocketSetWriteHandler,
    .ae_handler = connSocketEventHandler
};
```

Meaning

Redis creates a socket driver object.

Like:

None

```
Need read? use connSocketRead

Need write? use connSocketWrite

Need new client? use connSocketAccept
```

Why This Design?

Clean modular architecture.

Can later support:

- TLS sockets
- Unix sockets
- Other transports

==========

PART 3 — Creating Event Loop

==========

```C/C++
server.el                                                    =
aeCreateEventLoop(server.maxclients+CONFIG_FDSET_INCR);
```

What Happens Here?

Redis allocates internal event loop object.

Internal Structure

C/C++

```
typedef struct aeEventLoop {
    int maxfd;
    aeFileEvent events[];
    aeFiredEvent fired[];
    void *apidata;
} aeEventLoop;
```

Meaning of Fields

`maxfd`

Largest registered file descriptor.

Used for optimization.

`events[]`

Stores registered handlers.

Example:

```
None
fd 6 → accept handler

fd 12 → client read handler

fd 20 → client write handler
```

`fired[]`

Stores ready events returned by epoll.

Example:

```
None
fd 12 readable

fd 20 writable
```

`apidata`

Linux epoll internal state.

Contains:

```
None
epfd

epoll_event[]
```

==========

PART 4 — epoll Creation

==========

Inside:

```
None
aeApiCreate()
```

Redis creates:

```
C/C++
epfd = epoll_create(...)
```

This gives kernel-managed event queue.

Meaning

Kernel now tracks:

```
Which sockets are ready

Which sockets have incoming data

Which sockets can write
```

==========

PART 5 — Listener Initialization

==========

```C/C++
initListeners();
```

Listener Struct

```C/C++
struct connListener {

   int fd[];

   int port;

   ConnectionType *ct;

};
```

Meaning

Each listener stores:

```None
Listening socket fds

Port number

Connection backend
```

Example

```
None
listener.port = 6379

listener.fd[0] = socket descriptor 6

listener.ct = CT_Socket
```

==========

PART 6 — Bind + Listen

==========

Inside:

```
C/C++
anetTcpServer(...)
```

Then:

```
C/C++
bind(socket, addr, port)

listen(socket, backlog)
```

Meaning

Redis asks OS:

```
None
Reserve port 6379

Queue incoming TCP requests
```

Then Important Line:

```
C/C++
anetNonBlock(fd)
```

Meaning

Listening socket becomes:

```
None
O_NONBLOCK
```

So accept() never blocks.

==========

PART 7 — Register Accept Handler

==========

```C/C++
aeCreateFileEvent(server.el, fd, AE_READABLE, accept_handler, listener);
```

Meaning

Redis tells epoll:

```None
When listening socket becomes readable,

it means new connection arrived.

Call accept_handler.
```

Why Readable?

For listening socket:

```
Readable = pending client connection available
```

==========

PART 8 — Infinite Event Loop Starts

==========

```
aeMain(server.el);
```

Code

```
while (!stop) {

   aeProcessEvents(...)

}
```

Redis now loops forever.

==========

PART 9 — Wait for Kernel Events

==========

Inside:

```C/C++
aeApiPoll()
```

calls:

```C/C++
epoll_wait(epfd, events, ...)
```

What Happens?

Kernel blocks efficiently.

CPU sleeps until:

```
None
New client connection

Client sent command

Socket ready to write

Timeout
```

No Busy Waiting

Unlike polling:

```
None
check fd1?

check fd2?

check fd3?
```

Redis waits efficiently.

==========

PART 10 — Kernel Returns Ready Events

==========

Suppose:

```
None
fd 6 = listening socket

fd 12 = client socket
```

Kernel returns:

```
None
fired[0] = fd 6 readable

fired[1] = fd 12 readable
```

Stored in:

```
None
eventLoop->fired[]
```

==========

PART 11 — Process Ready Events

==========

```C/C++
for each fired event:

   fd = fired[j].fd

   fe = events[fd]
```

Redis maps ready fd → registered handler.

Example

If:

```None
fd 6 = listener
```

Then:

```None
rfileProc = accept_handler
```

Redis calls:

```C/C++
fe->rfileProc(...)
```

==========

PART 12 — Accepting New Clients

==========

Handler:

```C/C++
connSocketAcceptHandler(...)
```

Inside:

```C/C++
cfd = accept(fd,...)
```

Meaning

OS returns new socket:

```
None
Listening socket = 6

New client socket = 15
```

Important

Listening socket remains alive.

Client gets separate fd.

Redis Logs:

```
None
Accepted 192.168.1.5:53021
```

==========

PART 13 — Register Read Handler for Client

==========

After accept:

```C/C++
acceptCommonHandler(...)
```

calls:

```C/C++
connSetReadHandler(conn, readQueryFromClient)
```

Meaning

Redis tells event loop:

```None
When client socket has incoming bytes,

call readQueryFromClient()
```

Internally

```C/C++
aeCreateFileEvent(server.el,      client_fd,      AE_READABLE,
ae_handler, conn);
```

So New Client is Now Fully Managed.

==========

PART 14 — Client Sends Command

==========

Suppose client sends:

```None
SET user Samarth
```

Kernel marks socket readable.

epoll_wait returns:

```None
fd 15 readable
```

Redis Calls Read Handler

```
None
readQueryFromClient()
```

Which calls:

```
None
connRead()

read()
```

Redis Reads Raw Bytes

```
None
*3

$3

SET

$4

user

$7

Samarth
```

==========

PART 15 — Command Execution

==========

Redis parses protocol:

```
None
Command = SET

Key = user

Value = Samarth
```

Then updates in-memory dictionary.

==========

PART 16 — Register Write Event

==========

To send response:

```
None
+OK
```

Redis may register writable event.

When socket writable:

```
None
connSocketWrite()
```

Runs.

==========

PART 17 — Why This Is Extremely Fast

==========

✔ One Thread

No context switching chaos.

✔ epoll Scales to Huge Clients

10k–100k connections possible.

✔ RAM Data

Commands operate in memory.

✔ Non-blocking

No waiting on slow sockets.

✔ Event Driven

Only process sockets that have work.

==========

PART 18 — Full Lifecycle Example

==========

None

```
Redis starts

↓

Port 6379 opened

↓

epoll_wait()

↓

Client connects

↓

accept()

↓

register read handler

↓

Client sends GET key

↓

socket readable

↓

readQueryFromClient()

↓

lookup key in RAM

↓
```

```
socket writable

↓

send response

↓

back to epoll_wait()
```

==========

PART 19 — Why Redis Beats Threaded Servers

==========

Traditional Server	Redis
1 thread/client	1 event loop
heavy RAM	low RAM
context switching	none

locks needed	minimal
poor scaling	excellent

==========

PART 20 — Interview Gold Answer

==========

If asked:

Why Redis is so fast?

Answer:

```
None
Redis stores data in RAM and uses a single-threaded

epoll-based event loop that only processes ready sockets,

avoiding thread overhead and lock contention.
```

Final Mental Model

```
None
Kernel watches all sockets.

Redis sleeps.

Kernel says: fd 15 ready.

Redis processes fd 15 fast.

Sleep again.

Repeat millions of times.
```

Final Summary

Redis internally uses modular socket abstractions, non-blocking listeners, epoll-based event polling, and a single-threaded event loop to efficiently accept connections, read commands, execute them in RAM, and send replies with extremely low overhead.